Weidner_Samuel_Final_Proj

2026-10

```
setwd("~/Documents/GitHub/HDS-Foundations_of_Data_Science/final_proj") # set wd for
 ↪  reading filenames
filenames <- list.files(getwd(),pattern = ".csv") # identify all csv files in the repo
print(filenames)
```

```
## [1] "counts.csv"             "ensembl_id_mapping.csv" "meta_data.csv"
```

```
proj_data <- list()

data_path = "~/Documents/GitHub/HDS-Foundations_of_Data_Science/final_proj/"
# Set out repo data path
proj_data$gene_mapping <- read.csv(paste0(data_path, filenames[2])) # read file with
 ↪  non-ensembl names for genes of interest
proj_data$counts <- read.csv(paste0(data_path,filenames[1])) %>% # Read in counts data
  dplyr::rename(gene_name = X) # set a descriptive column name for the genes
proj_data$metadata <- read.csv(paste0(data_path,filenames[3])) # Read in metadata for
 ↪  covariates

proj_data$counts_v2 <- proj_data$counts %>% # join counts matrix to the mapping so we
 ↪  have the regular gene names
  dplyr::rename(ensembl_id = gene_name) %>%
  inner_join(proj_data$gene_mapping, by = 'ensembl_id') %>%
  select(-ensembl_id) %>%
  select(last_col(), everything())
```

```
gene_summary_func <- function(gene, gene_data){ # custom function for gene summary

  gene_data_subset <- gene_data %>%
    filter(gene_name == gene) # Subset data to our gene of interest

  # Compute mean, median, max, min, std dev, and IQR
  gene_summary <- gene_data_subset %>%
   summarise( # Use tidy summarize across our rows
    mean = mean(c_across(-gene_name)), # c across allows us to use the tidy function here
     ↪  without reformatting out data
    median = median(c_across(-gene_name)),
    sd = sd(c_across(-gene_name)),
    max = max(c_across(-gene_name)),
    min = min(c_across(-gene_name)),
    IQR = IQR(c_across(-gene_name))
  )

  return(gene_summary)
```

```
}

covariate_summary_func <- function(metadata, covariate){ # custom function for covariates

  covariate_summary <- metadata %>%
    select(!!sym(covariate)) %>% # subset to the covariate of interest using char
     ↪  parameter symbol to find column
    summarise(
      mean = mean(!!sym(covariate), na.rm = T),
      median = median(!!sym(covariate), na.rm = T),
      sd = sd(!!sym(covariate), na.rm = T),
      max = max(!!sym(covariate), na.rm = T),
      min = min(!!sym(covariate), na.rm = T),
      IQR = IQR(!!sym(covariate), na.rm = T)
    )

  return(covariate_summary)
}

selected_gene = "TP53" # Allow us to change gene as needed
selected_gene2 <- "BRCA1"
covariate_to_summarize <- "age_at_index" # allows to change as needed

summary_tp53 <- gene_summary_func(selected_gene, proj_data$counts_v2)
summary_brca1 <- gene_summary_func(selected_gene2, proj_data$counts_v2)
summary_age<- covariate_summary_func(proj_data$metadata, covariate_to_summarize)

print(summary_tp53)
```

```
##       mean median       sd   max min     IQR
## 1 4935.745 4441.5 3034.276 25897 274 3474.25
```

```
print(summary_brca1)
```

```
##       mean median       sd  max min     IQR
## 1 1304.207  919.5 1136.484 8050  21 1052.25
```

```
print(summary_age)
```

```
##       mean median       sd max min IQR
## 1 58.28049     58 13.27773  89  26  19
```

```
# Bind the rows together and add the Gene names back
combined_summary_genes <- bind_rows( # combine this so it is easier for input into our
 ↪  latex table
  mutate(summary_tp53, Gene = "TP53"),
  mutate(summary_brca1, Gene = "BRCA1")
) %>%
  select(Gene, everything()) # Reorder to put Gene first

# Generate Latex code
```

```r
# Generate the LaTeX table
latex_code_genes <- combined_summary_genes %>%
  kbl(
    format = "latex",
    booktabs = TRUE,
    digits = 2, # Round numbers to 2 decimal places
    caption = "Summary Statistics for TP53 and BRCA1 Expression Across Samples",
    label = "gene_stats",
    col.names = c("Gene", "Mean", "Median", "Std. Dev.", "Max", "Min", "IQR")
  ) %>%
  kable_styling(latex_options = c("hold_position")) # sets it so table appears where put
    ↪  it in the latex code instead of moving elsewhere in the output

latex_code_age <- summary_age %>%
  kbl( # create latex table code
    format = "latex",
    booktabs = TRUE,
    digits = 2, # Round numbers to 2 decimal places
    caption = "Summary Statistics for Cohort Age",
    label = "age_stats",
    col.names = c("Mean", "Median", "Std. Dev.", "Max", "Min", "IQR")
  ) %>%
  kable_styling(latex_options = c("hold_position")) # sets it so table appears where put
    ↪  it in the latex code instead of moving elsewhere in the output


# Print latex code to use in overleaf
print(latex_code_genes)
```

```
## \begin{table}[!h]
## \centering
## \caption{\label{tab:gene_stats}Summary Statistics for TP53 and BRCA1 Expression Across Samples}
## \centering
## \begin{tabular}[t]{lrrrrrr}
## \toprule
## Gene & Mean & Median & Std. Dev. & Max & Min & IQR\\
## \midrule
## TP53 & 4935.74 & 4441.5 & 3034.28 & 25897 & 274 & 3474.25\\
## BRCA1 & 1304.21 & 919.5 & 1136.48 & 8050 & 21 & 1052.25\\
## \bottomrule
## \end{tabular}
## \end{table}
```

```r
print(latex_code_age)
```

```
## \begin{table}[!h]
## \centering
## \caption{\label{tab:age_stats}Summary Statistics for Cohort Age}
## \centering
## \begin{tabular}[t]{rrrrrr}
## \toprule
## Mean & Median & Std. Dev. & Max & Min & IQR\\
```

```
## \midrule
## 58.28 & 58 & 13.28 & 89 & 26 & 19\\
## \bottomrule
## \end{tabular}
## \end{table}
```

```
selected_gene2 <- "BRCA1" # Set gene to plot first gene against
covariate <- "prior_treatment" # Set covariate

gene_visualize_func <- function(gene1, gene2, gene_data, metadata, covariate = NULL){

  gene1_data_subset <- gene_data %>%
    filter(gene_name == gene1) # Build gene1 data for histogram

  gene1_2_data_subset <- gene_data %>%
    filter(gene_name %in% c(gene1, gene2)) %>% # Subset gene data to two genes of
     ↪  interest from scatter
    pivot_longer( # Pivot format so it can be used easily in ggplot scatter
      cols = -gene_name,
      names_to = "sample_id",
      values_to = "expression_value"
    ) %>%
    mutate(
      expression_value = log2(expression_value + 1) # scale expression data with log2 so
       ↪  trends can be visualized more easily
    ) %>%
    pivot_wider(
      names_from = gene_name,
      values_from = expression_value
    )

  # Pivot for plotting

  gene1_long_subset <- gene1_data_subset %>%
  pivot_longer( # Pivot to a gene 1 long dataset for plotting and easy joining to
   ↪  metadata covariate
    cols = -gene_name,
    names_to = "sample",
    values_to = "expression_value"
  ) %>%
  mutate(sample_id_clean = gsub("\\.", "-", sample), # Need to reformat the sample ID so
   ↪  it matches our metadata ID names
         expression_value = log2(expression_value + 1)) # scale expression data with log2
          ↪  so trends can be visualized more easily

  # Add the metadata covariate

  if (is.null(covariate)) { # Check for covariate being set or not
  plotting_data_gene1 <- gene1_long_subset
  } else {
  merging_data <- metadata %>%
    select(barcode, !!sym(covariate)) %>% # Need to find the column symbol from the
     ↪  provided covariate string
    dplyr::rename(sample_id_clean = barcode, # Rename so it can be joined easily to our
↪  gene expression data
```

```
        covariate_value = !!sym(covariate)) # Set name of the covariate column

plotting_data_gene1 <- gene1_long_subset %>%
  left_join(merging_data, by = "sample_id_clean")
} # Join gene1 expression data with covariate for plotting

# Build Histogram
histogram <- ggplot(plotting_data_gene1, aes(x = expression_value)) +
  geom_histogram(
    bins = 10, # Set bin size
    fill = "darkgreen",
    color = "black",
    alpha = 0.7 # Set alpha for transparency
  ) +
  labs( # set labeling
    title = paste("Expression:", gene1),
    x = "Expression Value",
    y = "Frequency"
  ) +
 theme_minimal(base_size = 8) +
  theme( # Format size of title and labels
    plot.title = element_text(
      hjust = 0.5,
      face = "bold",
      size = 8
    ),
    axis.title = element_text(face = "bold"), # Bold axis titles
    panel.grid.minor = element_blank() # Remove extra gridlines
)

# Build Scatter Plot

scatter_plot <- ggplot(
  gene1_2_data_subset,
    aes(
      x = .data[[gene1]], # Set x and y for scatter
      y = .data[[gene2]]
    )
) +
  geom_point(
    color = "darkgreen",
    alpha = 0.6, # Set transparency for better readability of data density
    size = 1.5
) +
geom_smooth(method = "lm", color = "blue", se = FALSE, linewidth = 1.2) + # add
 ↪  trendline
stat_poly_eq( # add trendline equation to plot
  aes(label = paste(..eq.label.., ..rr.label.., sep = "~~~")),
  formula = y ~ x, # calculates formula and R^2 value for plot
  parse = TRUE,
  size = 3
) +
  labs(
    title = paste(gene1, "vs", gene2), # Set title
```

```
        x = gene1,
        y = gene2
  ) +
  theme_minimal(base_size = 8) +
  theme( # Set formatting for label and text
    plot.title = element_text(
      hjust = 0.5,
      face = "bold",
      size = 8
    ),
    axis.title = element_text(face = "bold"), # Bold axis titles
    panel.grid.minor = element_blank() # Remove extra gridlines
  )

  # Build Violin Plot

  covariate_string <- str_replace_all(covariate, "_", " ") # Replace underscores in
↪  covariate string so it prints as a phrase on the violin/boxplot

  violin_plot <- ggplot(
  plotting_data_gene1,
  aes(
    x = expression_value,
    y = covariate_value  # Set covariate to measure by
  )
) +
  geom_violin( # Set theme formats
    fill = "darkgreen",
    color = "black",
    alpha = 0.7,
    scale = "width"
  ) +
  geom_boxplot( # add boxplot also
    width = 0.15,          # thinner width so it is inside violin plot
    fill = "white",        # different fill from violin for contrast
    color = "black",
    alpha = 0.6,
    outlier.shape = NA     # don't include outlier points
  ) +
  labs( # Set labeling
    title = paste(gene1, "by", covariate_string),
    x = "Expression Value",
    y = covariate_string
  ) +
  theme_minimal(base_size = 8) +
  theme( # Set formatting for label and text
    plot.title = element_text(
      hjust = 0.5,
      face = "bold",
      size = 8
    ),
    axis.title = element_text(face = "bold"),
    axis.text.y = element_text(size = 7),
    panel.grid.minor = element_blank()
```

```
  )

  grid.arrange( # Put all plots on the same plane, two columns
    histogram,
    scatter_plot,
    violin_plot,
    ncol = 2
  )
}

png("gene_plots.png", width = 6, height = 4.5, units = "in", res = 300) # Set filename
 ↪  and sizing for output to upload to latex

gene_visualize_func(selected_gene, selected_gene2, proj_data$counts_v2,
                                          proj_data$metadata, covariate) # Display plots
 ↪  for our genes and covariate

dev.off() # output png
```

```
## pdf
##   2
```

```
set.seed(456) # Set seed for reproducibility of the gene sample

selected_genes_heatmap <- proj_data$counts %>% # Sample a set of genes for our heatmap
  dplyr::slice_sample(n = 10)

heatmap_genes_list <- selected_genes_heatmap$gene_name # create list for function input

covariate_heatmap <- "prior_treatment" # set covariate

gene_heatmap_func <- function(gene_list, gene_data, metadata, covariate){

  covariate_string <- str_replace_all(covariate, "_", " ") # Replace underscores in
 ↪  covariate string so it prints as a phrase on the heatmap

  gene_heatmap_data_subset <- gene_data %>%
    filter(gene_name %in% gene_list) # filter data to the listed genes


  rownames(gene_heatmap_data_subset) <- gene_heatmap_data_subset[[1]] # set rownames for
   ↪  matrix in heatmap

  gene_heatmap_data_subset <- gene_heatmap_data_subset %>%
    select(-gene_name) # remove the genename so we have a clean matrix for the heatmap

  # Get and sort covariate data by the covariate value
  covariate_ordered <- metadata %>%
    select(barcode, !!sym(covariate)) %>%
    dplyr::rename(sample_id = barcode, covariate_value = !!sym(covariate)) %>%
    mutate(sample_id_clean = gsub("\\-", ".", sample_id)) %>%  # clean up the sample id
     ↪  name so it matches the counts data
    select(sample_id_clean, covariate_value) %>%
```

```
    # Filter to only keep metadata for samples that actually exist in the matrix columns
    filter(sample_id_clean %in% colnames(gene_heatmap_data_subset)) %>%
    # set order for annotation bar
    arrange(covariate_value)

  # Reorder the matrix columns to match the sorted metadata
  gene_heatmap_data_subset <- gene_heatmap_data_subset %>%
    select(all_of(covariate_ordered$sample_id_clean)) # selecting by the ordered
     ↪  covariate data reorders the data here so it matches

  # Create annotation object matched to the new order sample id
  annotation_data <- covariate_ordered %>%
    select(covariate_value) # select the value alone

  rownames(annotation_data) <- covariate_ordered$sample_id_clean # set rownames

  covariate_levels <- unique(na.omit(annotation_data$covariate_value)) # Get unique
↪  non-null covariate categories

  # Generate named colors for covariate to use in the heatmap annotation label with
   ↪  circlize
  covariate_colors <- setNames(
    circlize::rand_color(length(covariate_levels)),
    covariate_levels
  )

  # Create annotation
  column_ha <- HeatmapAnnotation(
    Covariate = annotation_data$covariate_value, # values to annotate
    col = list(
      Covariate = covariate_colors # Set colors for annotation
    ),
    na_col = "grey", # set NA covariate values to grey
    annotation_legend_param = list( # add size of legend
      Covariate = list(
        title = covariate_string,
        title_gp = grid::gpar(fontsize = 6, fontface = "bold"),
        labels_gp = grid::gpar(fontsize = 7),
        grid_width = grid::unit(0.2, "cm"),
        grid_height = grid::unit(0.2, "cm")
      )
    )
  )

  # scale data with log2 because the selected genes have very different expression
   ↪  levels, making it hard to interpret

  gene_heatmap_matrix <- log2(
    as.matrix(gene_heatmap_data_subset) + 1 # use a log2 scale to make proportional
     ↪  changes look linear
  )

  Heatmap( # run heatmap function with our matrix of data
    as.matrix(gene_heatmap_matrix),
```

```
    name = "Expression Values",
    width = unit(4.5, "cm"),
    height = unit(5, "cm"),
    top_annotation = column_ha, # add annotation data
    show_row_names = TRUE, # shows genes on the rows
    show_column_names = FALSE, # don't show columns, because we have over 1200 samples
    cluster_columns = FALSE, # similarly don't cluster samples due to amount of samples
    cluster_rows = TRUE,
    column_title = "Samples",
    row_title = "Genes",
    heatmap_legend_param = list(
      title = "Count",
      title_gp = grid::gpar(fontsize = 8, fontface = "bold"),
      labels_gp = grid::gpar(fontsize = 7),
      direction = "horizontal"
      )
    )
}

png("heatmap.png", width = 6, height = 4.5, units = "in", res = 300) # Set filename and
 ↪  sizing for heatmap output to upload to latex


gene_heatmap_func(heatmap_genes_list, proj_data$counts,
                  proj_data$metadata, covariate_heatmap) # run function with our data


dev.off() # output png
```

```
## pdf
##   2
```

```
# Expression values matrix setup
counts_df <- proj_data$counts
gene_names <- counts_df$gene_name

counts_matrix <- as.matrix(counts_df %>% select(-gene_name)) # have to remove the name
 ↪  column because the statistical analysis and plotting needs just a matrix of
 ↪  expression values
rownames(counts_matrix) <- gene_names # set rownames

# Metadata setup (map covariate to 'treatment')
covariate <- "prior_treatment"  # Set covariate

metadata <- proj_data$metadata %>%
  dplyr::rename(
    id = barcode,
    treatment = !!sym(covariate)  # Rename to 'treatment' to match later code
  ) %>%
  mutate(sample_id_clean = gsub("\\-", ".", id)) %>% # clean id to match counts data
  select(-id) %>%
  filter(treatment %in% c("Yes","No")) %>% # Filter down to the two covariate groups of
   ↪  interest
```

```
  as.data.frame()

rownames(metadata) <- metadata$sample_id_clean

# Filter and align expression counts matrix to match the filtered down metadata samples
 ↪  that have our groups of interest
counts_matrix <- counts_matrix[, rownames(metadata), drop = FALSE] # keep only columns
 ↪  that exist in our clean metadata in the same order

# check to make sure previous steps matched the data order or throw error
stopifnot(identical(rownames(metadata), colnames(counts_matrix)))

# set treatment group as a factor with two groups
metadata$treatment <- factor(
  trimws(as.character(metadata$treatment)),
  levels = c("Yes", "No")
)

# create a container that has all of the data needed for the differential expression
 ↪  analysis
dds <- DESeqDataSetFromMatrix(
  countData = counts_matrix,
  colData = metadata,
  design = ~ treatment # set the design we want (mapping outcome (counts) vs. treatment
   ↪  variable that says if a patient has prior treatment or not)
)

dds <- DESeq(dds) # run the actual differential expression analysis statistical test to
 ↪  generate  values and fold changes

res <- results(dds, contrast = c("treatment", "Yes", "No")) # pull out the results, set
 ↪  the numerator to Yes and denominator to No, this means if you see a positive
 ↪  (upregulation) fold change then prior treatment led to higher expression and vice
 ↪  versa with a downregulation (negative fold change)
results_df <- as.data.frame(res)
results_df$gene_name <- rownames(results_df) # add rownames

# 4. Filter and Prep Data for Volcano Plot
volcano_data <- results_df %>%
  filter(!is.na(padj), !is.na(log2FoldChange)) # remove results that were null

png("volcano_plot.png", width = 7, height = 8, units = "in", res = 300) # set output for
 ↪  latex png

# 5. Generate Enhanced Volcano Plot
EnhancedVolcano(
  volcano_data,
  lab = volcano_data$gene_name,
  x = "log2FoldChange",
  y = "padj",  # Using adjusted p-values for graph

  # Significance thresholds
  pCutoff = 0.05,
  FCcutoff = 1.0,
```

```
  # Labels and formatting
  title = "Differential Gene Expression",
  subtitle = "No Prior Treatment vs Prior Treatment",
  xlab = bquote(~Log[2] ~ "Expression Value Change"),
  ylab = bquote(~-Log[10] ~ "Adjusted P-value"), # Explicitly labeling p value as
   ↪  adjusted

  # Formatting
  pointSize = 2.0,
  labSize = 4.0,

  # Formatting to prevent overlapping text
  drawConnectors = TRUE,
  widthConnectors = 0.5,
  colConnectors = 'black',
  max.overlaps = 20
)

dev.off() # output
```

```
## pdf
##   2
```